

CohortRound: Sensitivity-Guided Joint Rounding for Fixed-Latency FPGA DSP

Anonymous Authors
Research Proposal for DAC

Abstract—Fixed-point compilers normally choose word lengths and round every signal independently. This misses a hardware-exploitable degree of freedom: errors introduced on reconvergent DSP paths can cancel at downstream outputs. We propose *CohortRound*, a compiler and FPGA mechanism that groups a few physically local, cycle-aligned quantizers and jointly selects their floor/ceil directions. A bounded-depth selector reads only short discarded-fraction prefixes and minimizes sensitivity-weighted group discrepancy. The compiler co-optimizes cohorts, word lengths, alignment, and selector cost, and rejects cohorts whose control or routing overhead exceeds their datapath savings. An initial four-term prototype exhaustively validates all 4,096 selector states and passes bit-true compiler and RTL tests. Post-route FPGA cost remains the decisive open gate.

Index Terms—FPGA, DSP, fixed point, word-length optimization, joint rounding, compiler

I. MOTIVATION AND RESEARCH QUESTION

FPGA word lengths directly affect LUTs, registers, DSP packing, routing, and energy. Existing word-length optimization (WLO) can assign per-variable formats under an output-error constraint [1], and static rounding-mode selection can reduce implementation cost [2]. Nevertheless, the runtime decision at each quantizer is still usually local: round-to-nearest (RTN) minimizes each error separately, not their downstream combination.

Consider quantizers whose outputs later reconverge in a butterfly, FIR/filter bank, matrix–vector product, or convolution reduction. Independent small errors may add with the same sign. Conversely, deliberately choosing a locally non-nearest grid point at one site can cancel errors from other sites. The new question is therefore: *can a compiler synthesize small, fixed-latency rounding cohorts whose controlled error correlation permits narrower FPGA datapaths at the same end-to-end error bound?*

A four-term example illustrates why this differs from local RTN. Suppose the discarded fractions are $(0.49, 0.49, 0.49, 0.49)$. Independent RTN rounds every term down and loses 1.96 output units. A joint policy rounds exactly two terms up, producing a group error of only 0.04. Individual errors become larger at two sites, but the quantity consumed by the DSP reduction becomes much more accurate. This opportunity is common whenever separately quantized values later reconverge. It is absent when errors reach unrelated outputs, have incompatible timing, or require excessive wiring; identifying that boundary is part of the compiler problem rather than an assumption of the method.

Prior filter work shapes roundoff noise by selecting static widths [3], while correlation models estimate noise already created by a datapath [4]. Controlled/dependent rounding provides joint-rounding foundations [5], [6], and DiscQuant applies discrepancy ideas to offline neural-network weights [7]. Stochastic rounding hardware instead makes local PRNG-driven decisions [8]. None directly synthesizes a sensitivity-aware, per-sample joint selector under FPGA pipeline, logic-depth, and physical-locality constraints.

II. COHORTROUND MECHANISM

This section presents the complete numerical, compiler, and hardware mechanism. The central observation is that a quantizer has two distinct design choices. Its word length and binary point are static compile-time choices, but, for each run-time value, the compiler-generated circuit may still choose either adjacent representable value. Conventional WLO optimizes the former while usually fixing the latter to an independent rule such as RTN. CohortRound exposes the latter as a coordinated, hardware-constrained design variable.

A. From Local Quantization Error to Output Discrepancy

1) *Floor–residue decomposition*: Let quantization site i have step $\Delta_i > 0$. Every real value can be written uniquely as

$$x_i = \Delta_i(n_i + a_i), \quad n_i = \lfloor x_i/\Delta_i \rfloor, \text{quad}_i \in [0, 1). \quad (1)$$

The integer n_i identifies the lower representable value and a_i is the normalized discarded fraction, or *residue*. A binary decision $r_i \in \{0, 1\}$ selects the lower or upper neighboring grid point:

$$\hat{x}_i = \Delta_i(n_i + r_i), \quad \delta_i = \hat{x}_i - x_i = \Delta_i(r_i - a_i). \quad (2)$$

Thus $r_i = 0$ means floor and $r_i = 1$ means ceil. For example, with $x_i = 5.49$ and $\Delta_i = 1$, floor has error -0.49 while ceil has error $+0.51$. Independent RTN chooses floor because it minimizes $|\delta_i|$.

Equation (1) also gives unambiguous signed semantics. For $x_i = -2.3$ and $\Delta_i = 1$, $n_i = -3$ and $a_i = 0.7$; floor produces -3 with error -0.7 , whereas ceil produces -2 with error $+0.3$. The RTL uses an arithmetic shift to obtain the floor and treats the discarded field as a nonnegative residue. Consequently, the compiler, bit-true interpreter, and RTL do not require different rules for positive and negative operands.

2) *Why local RTN is not output-optimal:* Local RTN solves $\min_{r_i} |\delta_i|$ independently. A DSP, however, consumes combinations of these errors. Consider four equal-step values whose residues are $(0.49, 0.49, 0.49, 0.49)$ and whose quantized outputs are later summed. Independent RTN floors all four values, giving

$$e_{\text{RTN}} = (-0.49 - 0.49 - 0.49 - 0.49)\Delta = -1.96\Delta. \quad (3)$$

A joint policy may ceil exactly two values and floor the other two, giving

$$e_{\text{joint}} = (0.51 + 0.51 - 0.49 - 0.49)\Delta = 0.04\Delta. \quad (4)$$

Two local errors are deliberately made slightly larger, yet the downstream quantity becomes much more accurate. CohortRound therefore minimizes the error seen by selected DSP outputs, not the errors of isolated intermediates.

3) *Sensitivity-weighted vector model:* Let C be a set of $k = |C|$ quantization sites and define

$$D_C = \text{diag}(\Delta_1, \dots, \Delta_k), \quad \delta_C = D_C(\mathbf{r}_C - \mathbf{a}_C). \quad (5)$$

Within an affine downstream region, the resulting error at q monitored outputs is exactly

$$\mathbf{e}_C = A_C D_C(\mathbf{r}_C - \mathbf{a}_C), \quad (6)$$

where column i of $A_C \in \mathbb{R}^{q \times k}$ is the transfer sensitivity from local error δ_i to the monitored outputs. For example, if

$$y_1 = 2x_1 + x_2, \quad y_2 = x_1 - x_2, \quad (7)$$

then

$$A_C = \begin{bmatrix} 2 & 1 \\ 1 & -1 \end{bmatrix}. \quad (8)$$

A diagonal matrix W normalizes output tolerances or assigns application importance. The infinity norm then protects the worst normalized monitored output. Given complete residues, the ideal per-sample joint decision is

$$\mathbf{r}_C^* = \arg \min_{\mathbf{r} \in \{0,1\}^k} \|W A_C D_C(\mathbf{r} - \mathbf{a}_C)\|_\infty. \quad (9)$$

Equal sensitivity is a useful special case but not the complete mechanism. If all columns of A_C are identical, only the number of ceil decisions matters. With unequal sensitivities, their identities matter. For example, let $y = 2x_1 + x_2$, let both steps be one, and let $a_1 = a_2 = 0.49$. Flooring both gives $e = -1.47$. Ceil only x_1 gives $e = 2(0.51) - 0.49 = 0.53$, whereas ceil only x_2 gives $e = 2(-0.49) + 0.51 = -0.47$. Both choices have one ceil, but the second has the smaller output error. A general CohortRound selector must therefore emit a lane-specific vector, not merely a correction count.

The first compiler targets exact affine, acyclic regions, for which A_C is obtained analytically by a backward sensitivity pass. A nonlinear region is admitted only when the compiler can derive a sound Jacobian interval plus a remainder bound. Sensitivities sampled from representative inputs are insufficient for a deterministic certificate and are not used as proof.

B. Prefix-Limited Fixed-Latency Selector

1) *Why a prefix is necessary:* Equation (9) is a mathematical oracle, not a practical run-time implementation. Routing every discarded bit from every source to a centralized unit could consume more logic, wire energy, and timing slack than

the narrower datapath saves. CohortRound therefore exposes only the b most significant bits of each normalized residue:

$$p_i = \lfloor 2^b a_i \rfloor, \quad qquad p_i \in \{0, \dots, 2^b - 1\}. \quad (10)$$

For example, with $b = 3$ and $p_i = 3$, the hardware knows only that $a_i \in [3/8, 4/8) = [0.375, 0.5)$. For a prefix vector $\mathbf{p}$, all compatible residues form the Cartesian cell

$$\mathcal{B}(\mathbf{p}) = \prod_{i \in C} \left[\frac{p_i}{2^b}, \frac{p_i + 1}{2^b} \right). \quad (11)$$

2) *Robust policy synthesis:* For each prefix cell, compile time chooses one decision vector that minimizes the worst error over every unseen residue suffix:

$$T_C(\mathbf{p}) = \arg \min_{\mathbf{r} \in \{0,1\}^k} \sup_{\mathbf{a} \in \mathcal{B}(\mathbf{p})} \|W A_C D_C(\mathbf{r} - \mathbf{a})\|_\infty. \quad (12)$$

For a small cohort this problem is solved exactly by enumerating the 2^k floor/ceil vectors and evaluating the affine worst case over the prefix box. The resulting finite map $T_C : \mathbf{p} \mapsto \mathbf{r}$ is minimized into LUTs, comparators, and small adders. Importantly, the FPGA does *not* solve Eq. (12) each cycle. Optimization happens once at compile time; run time is a bounded-depth combinational lookup followed, if necessary, by a registered output.

The machine-checkable deterministic certificate of the resulting circuit is

$$\Gamma_C = \max_{\mathbf{p}} \sup_{\mathbf{a} \in \mathcal{B}(\mathbf{p})} \|W A_C D_C(T_C(\mathbf{p}) - \mathbf{a})\|_\infty. \quad (13)$$

This bound covers every prefix and every unobserved suffix. It is therefore not an average MSE, a distributional claim, or a result that holds only on a test set. It can be composed with other compiler error bounds.

3) *Closed-form equal-sensitivity bound:* For k equal-sensitivity sites with common step Δ , the group error is

$$e_C = \Delta \left(\sum_i r_i - \sum_i a_i \right). \quad (14)$$

With complete residues, choose the number of ceils as

$$m = \sum_i r_i = \text{round} \left(\sum_i a_i \right). \quad (15)$$

Then $|e_C| \leq \Delta/2$, while independent RTN can make all local errors have the same sign and approach $k\Delta/2$. With only prefixes, use the midpoint

$$\tilde{a}_i = (p_i + 1/2)2^{-b}, \quad m = \text{round} \left(\sum_i \tilde{a}_i \right). \quad (16)$$

The rounding of the midpoint sum contributes at most $1/2$, and each unknown suffix contributes at most $2^{-(b+1)}$. By the triangle inequality,

$$\begin{aligned} \left| m - \sum_i a_i \right| &\leq \left| m - \sum_i \tilde{a}_i \right| + \left| \sum_i (\tilde{a}_i - a_i) \right| \\ &\leq \frac{1}{2} + \frac{k}{2^{b+1}}, \end{aligned} \quad (17)$$

and hence

$$|e_C| \leq \Delta \left(\frac{1}{2} + \frac{k}{2^{b+1}} \right). \quad (18)$$

The implemented $k = 4, b = 3$ instance therefore has a 0.75Δ bound. It receives only $4 \times 3 = 12$ residue bits and emits a correction count $m \in \{0, 1, 2, 3, 4\}$, which needs three bits. This scalar implementation is an efficient special case; unequal or multi-output sensitivities require the lane-specific policy of Eq. (12).

4) *Information–hardware trade-off*: Prefix width b is a compiler variable rather than a fixed constant. A larger b narrows each cell, usually reducing Γ_C and enabling more aggressive word-length reduction. It simultaneously increases source fanout, selector inputs, wire count, truth-table complexity, and possibly logic depth. The same trade-off applies to cohort cardinality k . A candidate that violates the cycle budget is either pipelined, with its alignment registers explicitly charged to the candidate, or rejected. Consequently, the numerical oracle is an upper bound on opportunity, not an assumption of free hardware.

C. Hardware-Constrained Discrepancy Rounding

The compiler algorithm is called Hardware-Constrained Discrepancy Rounding (HCDR). It converts the per-cell rule above into profitable, scheduled FPGA structures through four stages.

1) Stage 1: numerical and implementation annotation:

The front end lowers fixed-point casts and explicit truncation sites into an IR that records source and destination formats, discarded-bit ranges, tie rules, and wrap or saturation semantics. A backward pass propagates a transfer vector from each site to each monitored output, producing A_C . Range analysis proves that overflow is absent or incorporates the chosen overflow semantics before a rounding certificate is accepted. Scheduling adds availability cycles, lifetimes, and initiation interval; a placement proxy adds physical regions, fanout estimates, and wire distances.

This stage deliberately distinguishes numerical compatibility from hardware compatibility. Two sites may have errors that cancel algebraically but be separated by many cycles or FPGA regions. Conversely, physically adjacent sites may affect unrelated outputs and offer no useful discrepancy reduction.

2) *Stage 2: candidate-cohort discovery*: HCDR builds a hypergraph whose vertices are quantization sites and whose hyperedges are candidate cohorts. Candidates are formed only when their members have compatible downstream influence, residue availability, lifetime, and physical span. A full-residue software oracle first estimates the maximum attainable bit saving. A candidate that cannot improve the error/width frontier even under the ideal oracle is discarded before hardware synthesis.

The discovery objective is not merely to maximize cohort size. Large cohorts offer more cancellation choices but grow the decision space and routing. HCDR therefore explores small local groups, alternative prefix widths, and, where useful, hierarchical cohorts. It retains negative candidates and rejection reasons so that evaluation exposes whether opportunity is limited by sensitivity, timing, or physical locality.

3) *Stage 3: selector synthesis and certification*: For each retained small candidate, exact enumeration or MILP constructs T_C and Γ_C under bounds on LUT depth, input fanout, pipeline latency, and placement span. Logic minimization exploits equivalent prefix states and, in the equal-sensitivity case, replaces a lane vector by a correction count. Larger candidates use a sensitivity-aware heuristic, but its error and hardware cost are reported against exact small-instance or decomposed oracles. HCDR emits both synthesizable RTL and a certificate artifact containing the prefix partition, chosen decisions, worst-case cells, and arithmetic semantics.

4) *Stage 4: joint extraction with WLO and scheduling*: The final pass chooses static formats $\mathbf{f}$, schedule S , prefix policies, and a non-overlapping subset of candidate cohorts. A representative objective is

$$\begin{aligned} \min_{\mathbf{f}, \mathbf{z}, \{T_C\}, S} \quad & C_{\text{data}}(\mathbf{f}, S) + \sum_C z_C C_C \\ C_C \triangleq \quad & C_{\text{sel}}(C, b, T_C) + C_{\text{route}}(C, b, S) \\ \text{s.t.} \quad & \Gamma_{\text{single}}(\mathbf{f}) + \sum_C z_C \Gamma_C \leq \epsilon, \\ & \Pi(S) \leq \Pi_{\text{max}}. \end{aligned} \quad (19)$$

Here z_C selects cohort C . C_{data} models the width-dependent cost of operators, registers, memories, and interconnect. C_{sel} models the lookup, correction, and optional pipeline logic. C_{route} models prefix wires, source fanout, physical span, and alignment registers. Additional constraints enforce timing, overflow safety, compatible cycle alignment, and at most one selected conflicting cohort per quantization site.

The additive certificate constraint is deliberately conservative: it remains valid when worst-case signs from different cohorts align. A future implementation may replace it with a tighter sign-aware polytope or affine-arithmetic composition, but may not weaken the guarantee by assuming independent noise. Most importantly, Eq. (19) permits $z_C = 0$. HCDR rejects a cohort when its selector and routing overhead exceed the datapath saving; joint rounding is an opportunity, not a mandatory transformation.

D. Generated Run-Time Datapath

For every selected cohort, the backend produces two paths. The main data path keeps the floor value at the newly selected narrow precision. A narrow metadata path carries b leading residue bits to the cohort selector. The selector emits either a per-lane ceil vector or, in the equal-sensitivity reduction special case, an aggregate correction. Required delays align the decision with the main data. Conceptually,

$$\begin{aligned} \text{wide value} &\longrightarrow \begin{cases} \text{narrow floor value,} \\ \text{short residue prefix,} \end{cases} \\ \text{prefix} &\longrightarrow T_C \longrightarrow \text{decision/correction,} \\ \text{floor value} + \text{decision} &\longrightarrow \text{corrected narrow result.} \end{aligned} \quad (20)$$

The main benefit is that wide low-order data bits need not remain live through all intervening registers, buffers, interconnect, and operators. Only short prefix metadata survives long enough to coordinate the rounding decision.

E. Scope, Boundary Cases, and Relation to Delayed Rounding

Joint rounding alone is not the claimed novelty. The proposed compiler/hardware abstraction is

$$\text{cohort} = (C, A_C, D_C, b, T_C, \Gamma_C, \text{schedule}, \text{placement window}). \quad (21)$$

which connects compile-time sensitivity and physical analysis to a per-sample, fixed-latency hardware policy. Conventional WLO changes static formats but does not synthesize correlated run-time decisions. Static rounding-mode search fixes one mode per site for all samples. Correlation analysis predicts noise already created by a datapath but does not choose its signs. Stochastic rounding is run-time but generally makes local PRNG-driven decisions rather than a sensitivity-conditioned joint decision. Dependent/discrepancy rounding supplies mathematical foundations but does not by itself discover dynamic DSP cohorts or map them to prefix-limited, physically constrained FPGA circuits.

Delayed rounding is the essential structural baseline. If all cohort members feed one immediate sum with a common step and equal sensitivity, retaining guard bits and rounding once after the sum may be both more accurate and cheaper. The current four-term correction-count unit demonstrates the arithmetic and RTL mechanism, but that simple pattern alone does not establish a distinct compiler contribution.

CohortRound becomes structurally different when values must be quantized before traversing separate branches, buffers, memories, or operators and later reconverge with unequal signed sensitivities or affect multiple outputs. Moving one common rounding point past such a region is not equivalent unless the implementation preserves wide values throughout every branch. CohortRound instead narrows those main paths early and transports only bounded prefix metadata. The decisive experiments must therefore include both immediate reductions and branched/reconvergent graphs, compare against full-precision late rounding, and account for every selector, alignment, and routing resource.

Finally, the first version does not claim unrestricted support. Exact affine acyclic regions are in scope. Saturating discontinuities require explicit case partitioning; nonlinear operators require certified derivative and remainder bounds; feedback filters require an invariant over data and rounding state. The stateless selector provides a per-invocation deterministic discrepancy bound, not probabilistic unbiasedness or bounded long-term drift. These limits define where the mechanism is sound and prevent empirical accuracy from being mistaken for a compiler guarantee.

III. PROTOTYPE EVIDENCE AND DAC EVALUATION

We added a `cohort_mac_reduce` intrinsic, a fail-closed canonical dot-product recognizer, bit-true interpreter semantics, Chisel lowering, and three matched RTL designs. All use 24 signed Fixed[16, 14] terms, the same digit-serial multiplier and control schedule, and a Fixed[20, 13] output. Only the rounding/reduction policy changes: CohortRound, independent

TABLE I
COMPLETED PRE-VIVADO GATES.

Gate	Result	Coverage
Prefix oracle	4,096/4,096	all 8^4 states
Bit-true datapath	2,000/2,000	random signed MACs
Compiler tests	7/7	checker, lowering, fail-closed
Chisel/Verilator	5/5	target and two baselines
RTL lint	3/3	all generated tops

per-product RNE, or full-precision accumulation followed by one RNE.

The current target forms exact products sequentially. For every group of four, it arithmetic-shifts each signed product to a 17-bit floor, collects the three most significant discarded bits, and accumulates the floors in 19 bits. The selector converts the four prefixes into a correction in $[0, 4]$, which is added to a 23-bit global accumulator. Negative products use the same floor-residue decomposition, so no sign-specific approximation is hidden in the software model. Two matched controls preserve input/output formats, operation count, multiplier, handshake, and schedule: one independently rounds every product, while the other accumulates in 38 bits and rounds once.

The lowering reduces the global accumulator from 38 to 23 bits and uses a 19-bit four-term group accumulator. However, the present serial multiplier still forms a full 32-bit product; therefore these results establish numerical and RTL feasibility, not a large total-area saving.

The tests deliberately cover more than average error. The Python selector exhausts all prefix states, including ties; an independently written integer oracle checks negative operands, extreme products, output wraparound, and 2,000 random vectors. RTL tests repeat exhaustive selector checking, verify the one-cycle valid/bubble protocol, and cross-check the complete 24-term machine and both baselines. Unsupported cohort sizes and noncanonical kernels fail closed instead of silently falling back to different arithmetic.

The full evaluation will cover FIR/polyphase filters, FFT butterflies, matrix-vector/beamforming, and convolution reductions, with negative controls that have no local alignable cohort. Baselines include truncation, independent RTN and stochastic rounding, static mode optimization [2], and width-only WLO [1]. We will report post-route LUT/FF/DSP/BRAM, Fmax, II, latency, energy/sample, routing/alignment overhead, error certificates, and exact-oracle gap. The idea passes only if at least three basic DSP kernels save at least one fractional bit at the same deterministic error bound, retain II, lose less than 5% Fmax, and achieve net post-route area or energy benefit. If the shared multiplier dominates, a truncated-product/high-product-plus-prefix lowering is required; changing the device or timing target is not evidence.

To isolate causality, the study uses four ablations: (i) identical widths with independent decisions, measuring the value of correlation; (ii) oracle cohorts without locality constraints, measuring the physical feasibility gap; (iii) identical cohorts with full residues versus short prefixes, measuring the information-routing trade-off; and (iv) compiler-selected

versus random or cardinality-matched cohorts, measuring the value of sensitivity analysis. Each design is synthesized under the same part, clock, II, I/O, and multiplier architecture. Power claims require workload-derived SAIF/VCD activity rather than vectorless estimates.

Three failure modes are reported explicitly. First, many kernels may expose only isolated quantizers or sensitivities that do not permit cancellation. Second, the selector and alignment network may consume more resources than one saved fractional bit. Third, a serial architecture may be multiplier-dominated, making reduction-width savings invisible at top level. The last case motivates a high-product-plus-prefix multiplier, but that extension must be evaluated as a separate mechanism and ablation rather than credited to cohort rounding.

A. Compiler integration and supported scope

The front end first lowers arithmetic casts and explicit quantization sites into an IR that records source/destination formats, discarded-bit ranges, and overflow semantics. A backward sensitivity pass annotates each site with its transfer vector to selected outputs. Scheduling then supplies availability cycles and lifetimes, while a placement proxy supplies candidate regions and wire-distance estimates. HCDR consumes these annotations and emits ordinary fixed-width operators plus a selector module, residue-prefix connections, and any required delay registers. The bit-true interpreter evaluates exactly the same prefix policy used by RTL, preventing a floating-point software oracle from silently defining different tie or signed-shift behavior.

The first compiler version deliberately targets real-valued affine regions and acyclic reductions. Saturation and overflow are orthogonal: range analysis must prove no overflow or model the selected wrap/saturate semantics before a rounding certificate is valid. Feedback filters require an invariant over both data state and rounding state and are therefore a separate extension. Likewise, the stateless selector provides a per-invocation deterministic discrepancy bound; it does not claim probabilistic unbiasedness. A future debt controller may reduce long-term drift only if finite-state reachability proves its debt is bounded for every legal input sequence.

The expected deliverables are consequently concrete: a cohort-aware numerical IR and discovery pass; an exact small-cohort robust selector algorithm with a scalable heuristic; machine-checkable error certificates that include unseen residue suffixes; and an RTL backend with post-route cost feedback. Releasing rejected candidates and their causes is also important: it reveals whether the limiting factor is numerical structure, schedule alignment, selector logic, or routing, and prevents reporting only favorable kernels.

IV. CONCLUSION

CohortRound turns runtime correlation among quantization errors into a compiler-visible FPGA design variable. Its novelty is not discrepancy rounding alone, but sensitivity-aware cohort discovery, fixed-latency prefix-selector synthesis, deterministic certification, and physical-cost-aware extraction.

The completed prototype supports the numerical and implementation mechanism; matched Vivado post-route results remain the decisive test of DAC-level impact.

REFERENCES

- [1] I. Winandy, A. Dion, F. Manni, P.-L. Garoche, D. B. Khalifa, and M. Martel, "Automated fixed-point precision optimization for FPGA synthesis," *IEEE Open Journal of Circuits and Systems*, vol. 6, pp. 192–204, 2025.
- [2] D. Menard, D. Novo, R. Rocher, F. Cathoor, and O. Sentieys, "Quantization mode opportunities in fixed-point system design," in *Proceedings of the 18th European Signal Processing Conference (EUSIPCO)*, Aalborg, Denmark, 2010, pp. 542–546.
- [3] G. A. Constantinides, P. Y. K. Cheung, and W. Luk, "Roundoff-noise shaping in filter design," in *Proceedings of the IEEE International Symposium on Circuits and Systems (ISCAS)*, vol. 4, 2000, pp. 57–60.
- [4] J.-C. Naud, D. Menard, G. Caffarena, and O. Sentieys, "A discrete model for correlation between quantization noises," *IEEE Transactions on Circuits and Systems II: Express Briefs*, vol. 59, no. 11, pp. 800–804, 2012.
- [5] L. H. Cox and L. R. Ernst, "Controlled rounding," *INFOR: Information Systems and Operational Research*, vol. 20, no. 4, pp. 423–432, 1982.
- [6] R. Gandhi, S. Khuller, S. Parthasarathy, and A. Srinivasan, "Dependent rounding and its applications to approximation algorithms," *Journal of the ACM*, vol. 53, no. 3, pp. 324–360, 2006.
- [7] J. Chee, A. Backurs, R. Heck, L. Zhang, J. Kulkarni, T. Rothvoss, and S. Gopi, "Discquant: A quantization method for neural networks inspired by discrepancy theory," in *Proceedings of the 38th Conference on Learning Theory*, ser. Proceedings of Machine Learning Research, vol. 291. PMLR, 2025, pp. 924–951. [Online]. Available: <https://proceedings.mlr.press/v291/chee25a.html>
- [8] M. Mikaitis, "Stochastic rounding: Algorithms and hardware accelerator," in *Proceedings of the International Joint Conference on Neural Networks (IJCNN)*. IEEE, 2021.